IMPORTANCIA DEL DISEÑO DE SOFTWARE

Fundación Universitaria Compensar – Ucompensar

Estudiante: Laura Camila León

Fecha: 18 de octubre de 2026

Escrito integrado

El desarrollo de software es una actividad compleja que va mucho más allá de escribir código. Antes de construir cualquier sistema, es indispensable pensar en su diseño, es decir, en cómo se organizarán sus componentes, cómo se comunicarán entre sí y cómo se garantizará que el producto final sea mantenible, escalable y comprensible para cualquier profesional que intervenga en él. El diseño de software cumple un papel análogo al de los planos en la construcción de un edificio: sin ellos, se puede levantar una estructura, pero difícilmente será segura, funcional y eficiente a largo plazo.

Cuando se desarrolla software sin un diseño previo, las consecuencias suelen manifestarse de forma progresiva y acumulativa. En las primeras etapas el sistema puede parecer funcional, pero a medida que crece en funcionalidades y usuarios, los problemas se vuelven evidentes. Entre los efectos adversos más comunes se encuentran el alto acoplamiento entre componentes, que significa que modificar una parte del sistema obliga a cambiar muchas otras; la baja cohesión, donde las clases y módulos asumen responsabilidades que no les corresponden; la duplicación de código, que dificulta las correcciones y genera inconsistencias; y la deuda técnica, que es el costo acumulado de tomar atajos durante el desarrollo. Según Díaz, Harari y Amadeo (2013), un software que no ha sido diseñado de forma centrada en el usuario y con una arquitectura clara tiende a generar productos difíciles de usar, costosos de mantener y propensos a fallos en producción. Además, la ausencia de diseño hace que la incorporación de nuevos desarrolladores al equipo sea un proceso lento y frustrante, ya que no existen referencias claras sobre cómo está organizado el sistema ni por qué se tomaron ciertas decisiones.

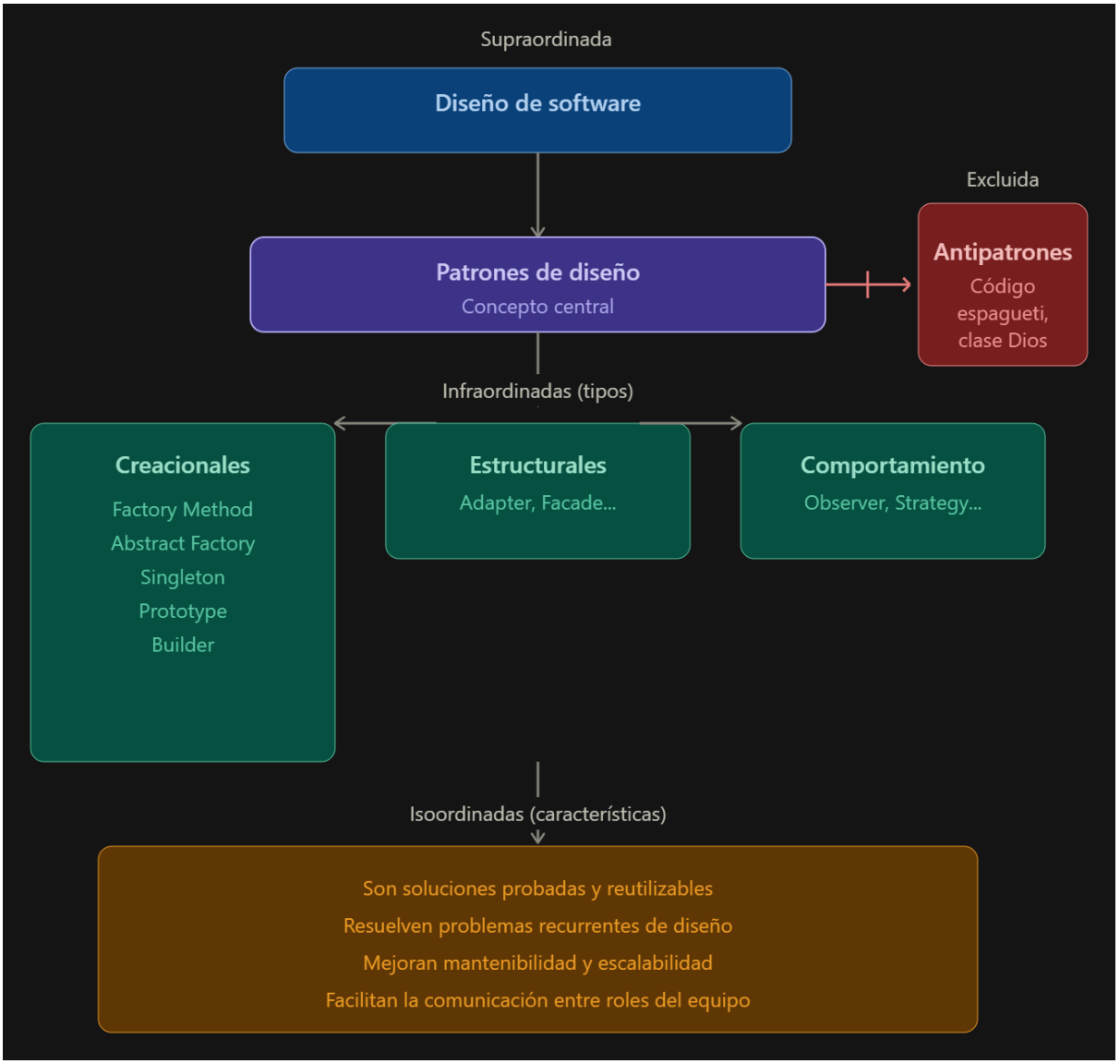
Es precisamente en este contexto donde los patrones de diseño adquieren su relevancia. Un patrón de diseño es una solución probada y reutilizable para un problema recurrente en el desarrollo de software. No se trata de código que se copia y pega, sino de un esquema conceptual que guía al desarrollador para resolver un problema de diseño de manera elegante y eficaz. Los patrones de creación, por ejemplo, abordan el problema de cómo instanciar objetos de forma flexible: el patrón Factory Method delega la creación de objetos a subclases para no depender de clases concretas; Abstract Factory permite crear familias de objetos relacionados sin especificar sus clases concretas; Singleton garantiza que una clase tenga una única instancia en todo el sistema; Prototype permite clonar objetos existentes en lugar de crearlos desde cero; y Builder separa la construcción de un objeto complejo de su representación, permitiendo crear diferentes configuraciones con el mismo proceso. Cada uno de estos patrones responde a una necesidad específica de diseño que, sin ellos, se resolvería con soluciones improvisadas que generan los problemas descritos anteriormente.

La relación entre los patrones de diseño y los distintos roles que puede desempeñar un ingeniero de software es directa y significativa. Un arquitecto de software utiliza los patrones como vocabulario

común para comunicar decisiones de diseño al equipo; un desarrollador los emplea para escribir código limpio y extensible; un analista de calidad los reconoce como indicadores de buenas prácticas; y un líder técnico los aplica para estandarizar las soluciones dentro de un proyecto. Besembel (2009) señala que las técnicas de diseño orientado a objetos permiten estructurar el desarrollo de manera que cada componente tenga responsabilidades bien definidas, lo cual facilita el trabajo colaborativo entre diferentes roles. Así, los patrones de diseño no son solo herramientas técnicas, sino que funcionan como un lenguaje compartido que mejora la comunicación, reduce la ambigüedad y eleva la calidad del producto final. En definitiva, diseñar antes de construir no es un lujo ni una etapa opcional: es la base que distingue al software profesional del software improvisado, y los patrones de diseño son el puente que conecta la teoría del buen diseño con la práctica cotidiana de la ingeniería de software.

Mentefacto conceptual

Como el mentefacto debe entregarse como imagen (.jpeg o .bmp), te describo su estructura para que lo elabores en una herramienta visual (PowerPoint, Canva, Draw.io o similar):



Relaciones del mentefacto:

- Supraordinada (arriba): El diseño de software contiene a los patrones de diseño como una de sus herramientas fundamentales.
- Infraordinadas (abajo-izquierda): Los patrones se subdividen en creacionales (Factory Method, Abstract Factory, Singleton, Prototype, Builder), estructurales y de comportamiento.
- Isoordinadas (abajo): Características esenciales que definen a los patrones.
- Excluida (derecha): Los antipatrones se distinguen de los patrones porque representan malas prácticas disfrazadas de soluciones; son lo opuesto conceptual.

REFERENCIAS

Besembel, I. (2009). TDSO: una técnica para el diseño de software orientado por objetos. *Revista Ciencia e Ingeniería*, 30(3), 193-200. https://elibro-net.ucompensar.basesdedatosezproxy.com

Debrauwer, L. (2018). *Patrones de diseño en Java: los 23 modelos de diseño*. Ediciones ENI.

Díaz, J., Harari, I. y Amadeo, A. P. (2013). *Guía de recomendaciones para diseño de software centrado en el usuario*. Editorial de la Universidad Nacional de La Plata. https://elibro-net.ucompensar.basesdedatosezproxy.com